Guía del proyecto

Aplicación síncrona y asíncrona con PyMongo y MongoDB Atlas

Dato	Detalle
Repositorio	https://github.com/Anngel-o/pymongo_sample_mflix
Base de datos	`sample_mflix`, colección `movies` (MongoDB Atlas, plan gratuito)
Materia	Cómputo paralelo y distribuido (8CC2)
Integrantes	Jose Angel Ortiz Meraz [353195]
	Luz Mariam Garcia Castillo [348409]
	Carlos Sanchez Fuentes [368058]
Lenguaje	Python 3.11 o superior
Librerías	PyMongo (`MongoClient` y `AsyncMongoClient`) y python-dotenv

1. Descripción del proyecto

Son dos aplicaciones de consola que administran las películas de la base `sample_mflix`: una **síncrona** (PyMongo) y una **asíncrona** (PyMongo Async con `AsyncMongoClient`). Las dos hacen lo mismo: **carga masiva**, **CRUD** (crear, leer, actualizar, eliminar) y **consultas**. Después se comparan sus tiempos.

El proyecto es el repositorio de GitHub. Esta guía sirve para entender cómo está hecho, cómo se configuró el entorno y cómo usarlo.

2. Estructura del repositorio

Archivo	Para qué sirve
`config.py`	Lee el `.env` y define los parámetros (tamaño de lote, concurrencia, etc.)
`app_sync.py`	Aplicación síncrona con menú
`app_async.py`	Aplicación asíncrona con menú
`comparar.py`	Ejecuta la prueba de rendimiento de las dos apps y compara tiempos
`test_conexion.py`	Prueba rápida de conexión a Atlas (síncrona y asíncrona)
`requirements.txt`	Librerías necesarias
`README.md`	Resumen del proyecto y cómo ejecutarlo
`REPORTE.md`	Reporte con la comparación entre las dos apps
`Guia_del_Proyecto.docx`	Esta guía

Archivo	Para qué sirve
`resultados.md`	Tabla de tiempos. **Se crea al ejecutar `comparar.py`**; antes de eso no existe
`.env`	Datos de conexión (contraseña). Es local y **no se sube a GitHub**

3. Funcionamiento

3.1 Las dos colecciones

- **`movies`**: la colección original (21,349 películas). **Solo se lee**: de aquí salen las consultas y los datos de la carga.
- **`movies_test`**: colección de pruebas. **Aquí se escribe**: se carga, se crea, se actualiza y se elimina.

Se separan para no dañar el dataset original si algo sale mal. Para dejar `movies_test` limpia se usa la opción 13 (la deja vacía) o la opción 1 (la borra y la vuelve a llenar). El código se niega a arrancar si las dos apuntan a la misma colección.

3.2 Síncrono vs. asíncrono

En la app **síncrona**, cada operación espera la respuesta de Atlas antes de pasar a la siguiente. En la **asíncrona** se usa `async`/`await`: mientras una operación espera la red, el programa avanza con otras. Por eso la asíncrona gana cuando hay **muchas operaciones independientes**, y con una sola operación no hay diferencia.

3.3 Carga masiva (opción 1)

- Se leen N películas de `movies` (por defecto 2,000; el usuario puede escribir otro número).
- Se borra `movies_test` y las películas se parten en lotes de 200.
- Cada lote se inserta con un `bulk_write()` de operaciones `InsertOne` (una sola petición por lote).
- **Síncrona**: un lote tras otro. **Asíncrona**: hasta 5 lotes a la vez (`TaskGroup` + `Semaphore`).

3.4 CRUD (opciones 2 a 6)

Letra	Operación	Función de PyMongo	Opción
C (Create)	Crear una película	`insert_one`	2
R (Read)	Buscar una película por título	`find_one`	3
U (Update)	Cambiar el año de una película	`update_one`	4
D (Delete)	Eliminar una película	`delete_one`	5
CRUD masivo	Hace crear, leer, actualizar y eliminar con varias películas inventadas	las cuatro anteriores	6

En el CRUD masivo, la síncrona procesa una película tras otra; la asíncrona procesa varias a la vez con `gather()` y `Semaphore`.

3.5 Consultas (opciones 7 a 11)

Se hacen sobre `movies`. Cada una muestra una función distinta de MongoDB:

Opción	Consulta	Qué se usa
7	Películas de un año	Filtro con `find`
8	Cuántas películas hay de un género	`count_documents`
9	Las 5 mejor calificadas	`sort` y `limit`
10	Los 5 géneros con más películas	`aggregate` (`$unwind`, `$group`)
11	Las consultas 7, 8, 9 y 10 juntas	En la asíncrona se lanzan con `gather()`

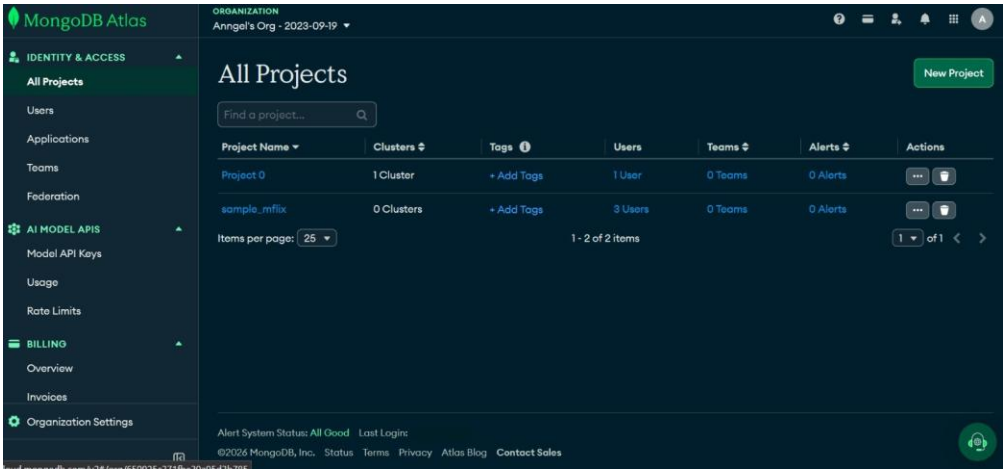
3.6 Herramientas de asyncio y bulk_write

Herramienta	Qué hace en este proyecto
`bulk_write()`	Manda un lote completo de inserciones en una sola petición
`asyncio.TaskGroup`	Agrupa las tareas de la carga masiva; espera a que todas terminen y, si una falla, cancela las demás
`asyncio.gather()`	Lanza varias operaciones juntas (CRUD masivo y consultas) y devuelve los resultados en orden
`asyncio.Semaphore`	Limita a 5 las operaciones simultáneas, para no saturar el clúster gratuito

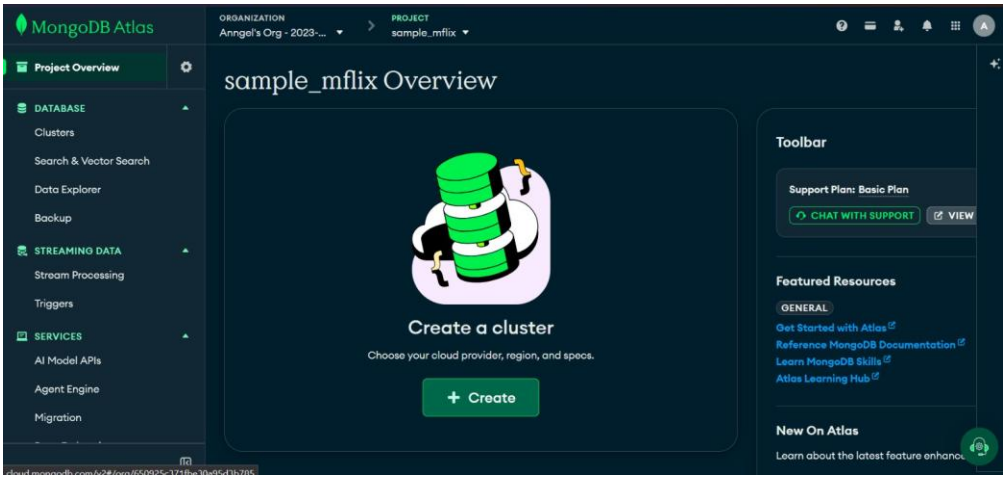
4. Configuración de MongoDB Atlas

Pasos que se siguieron, con capturas de pantalla.

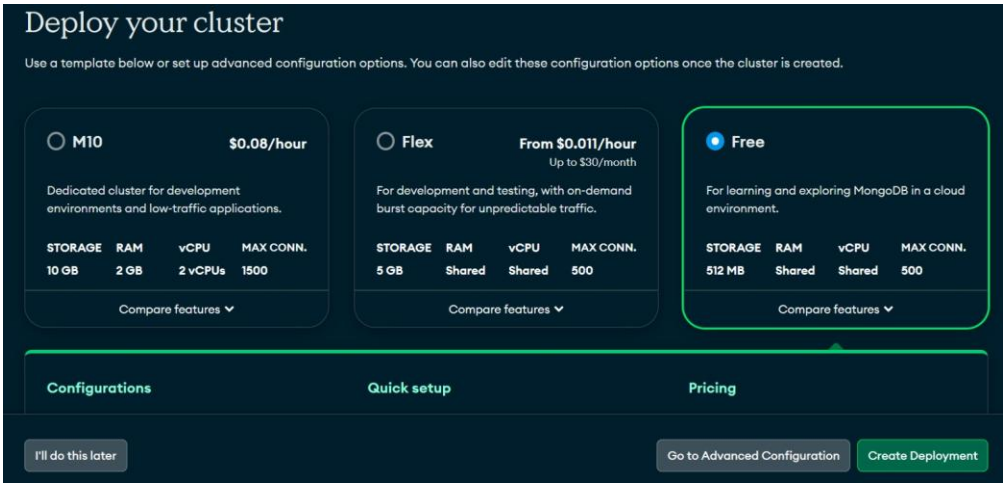
1. Entrar a MongoDB Atlas y crear un proyecto llamado `sample_mflix`.



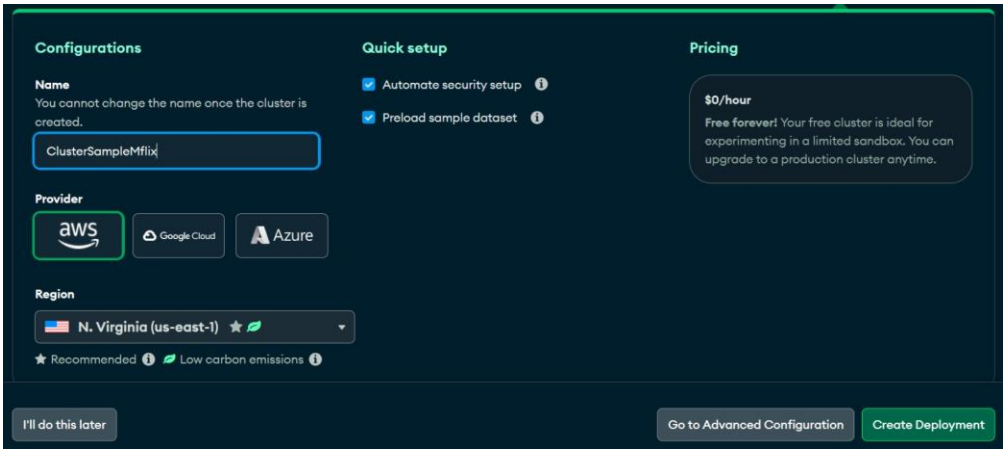
2. Dentro del proyecto, crear un clúster (botón **Create**).



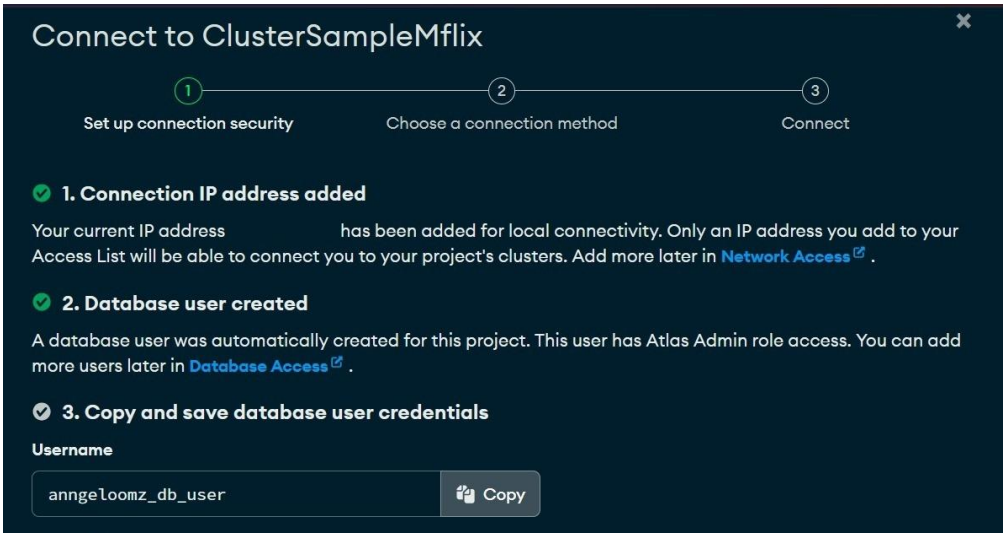
3. Elegir el plan **Free** (512 MB, sin costo).



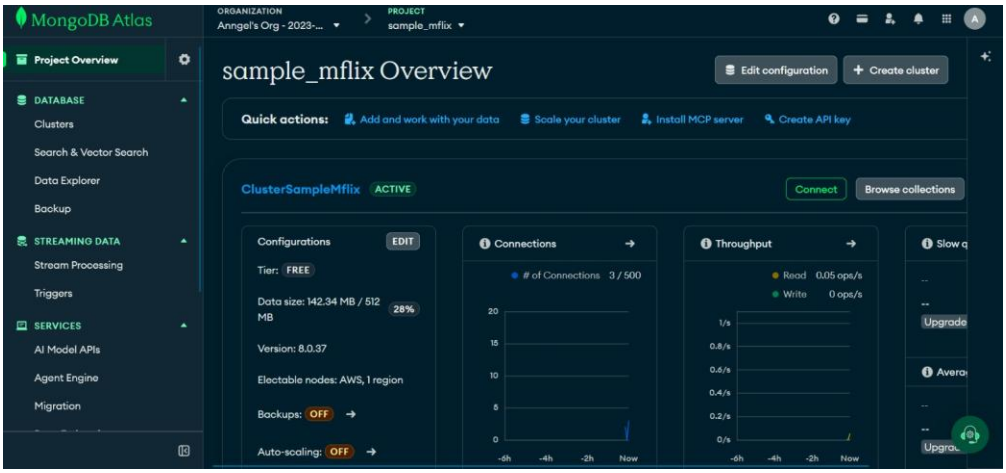
4. Nombrar el clúster (`ClusterSampleMflix`), elegir AWS y la región N. Virginia. Marcar **Automate security setup** y **Preload sample dataset**: esto carga el dataset `sample_mflix`.



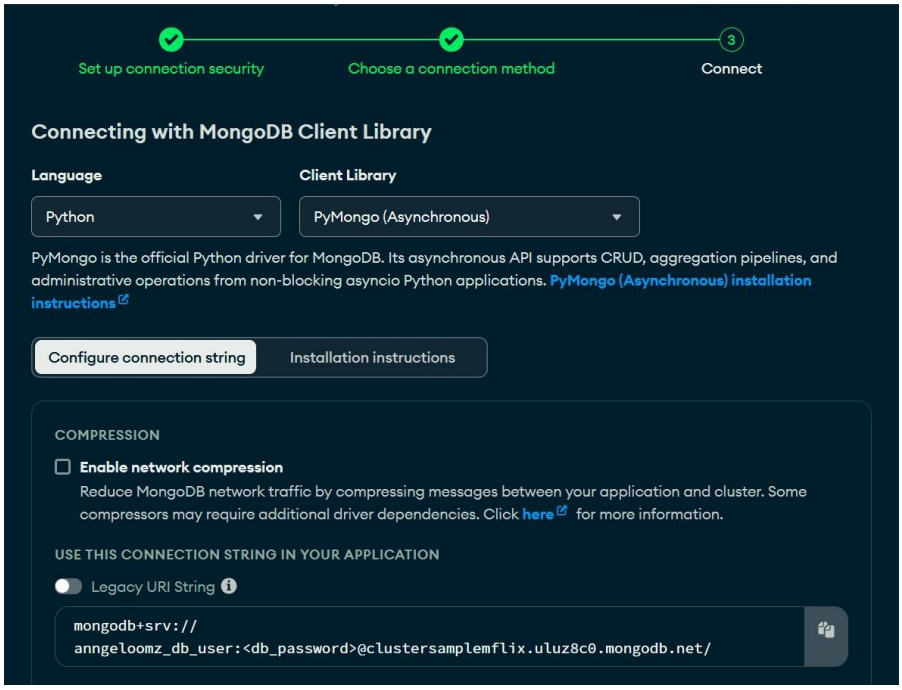
5. Atlas crea un usuario de base de datos con rol Atlas Admin y agrega tu IP actual. **Guarda la contraseña**: no se vuelve a mostrar.



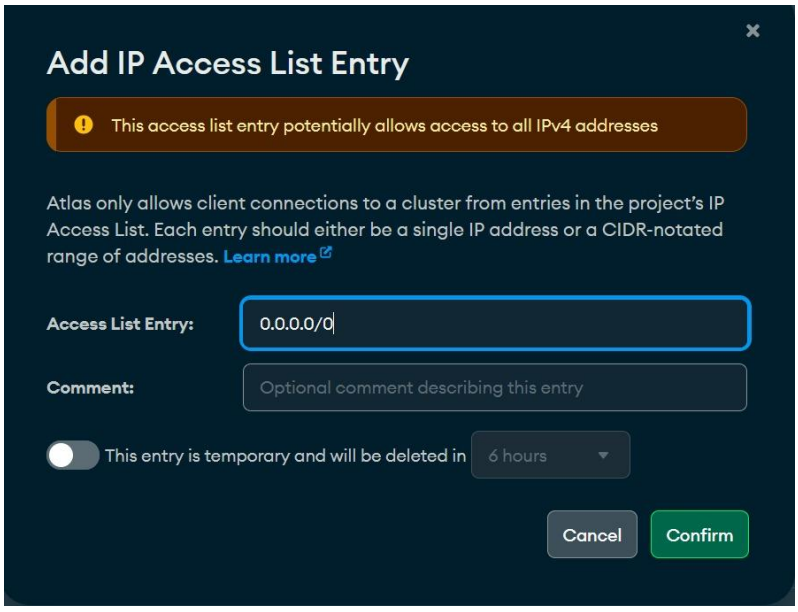
6. En el Overview del proyecto, el **Data size** de unos 142 MB confirma que el dataset ya se cargó.



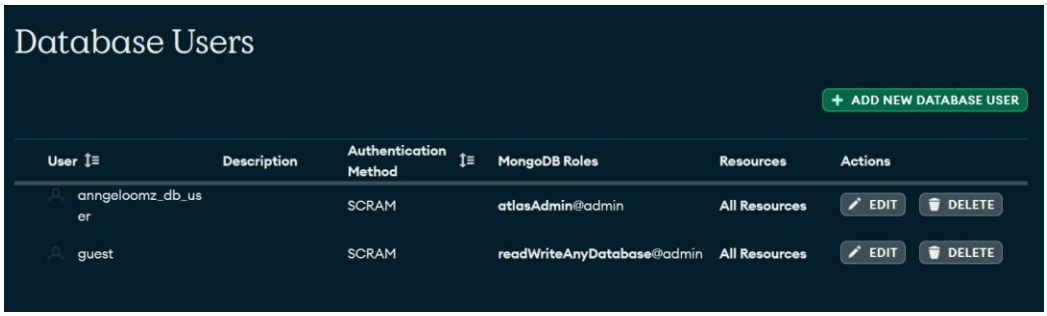
7. Obtener la cadena de conexión: **Connect** → **Drivers** → **Python (PyMongo)**. La cadena tiene la forma `mongodb+srv://usuario:<db_password>@cluster.mongodb.net/`; hay que cambiar `<db_password>` por la contraseña real.



8. En **Network Access** → **IP Access List**, agregar `0.0.0.0/0` para que cualquier integrante del equipo pueda conectarse. Se recomienda hacerlo temporal o borrarlo al terminar el proyecto.



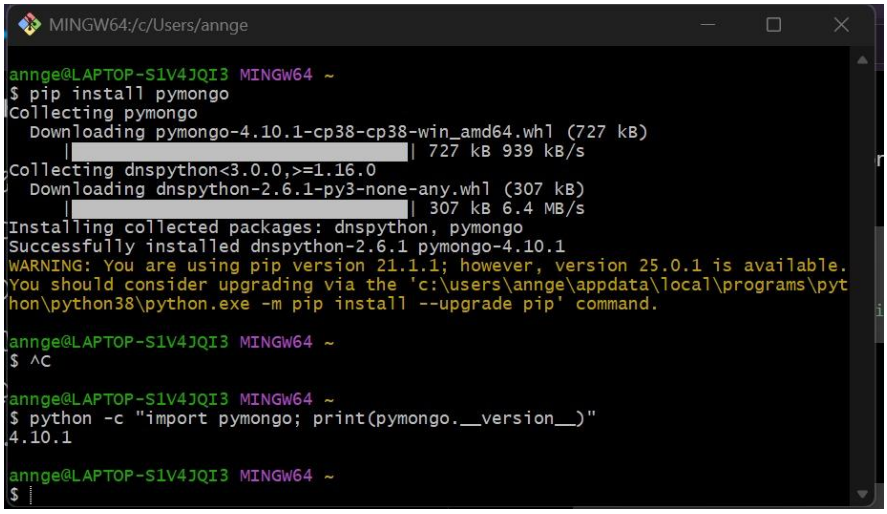
9. En **Database Access**, crear un usuario invitado (`guest`) con rol `readWriteAnyDatabase` para las conexiones del equipo. Usa una contraseña segura, no igual al nombre de usuario. El usuario administrador es solo para el dueño del proyecto.



10. El entorno de MongoDB queda listo.

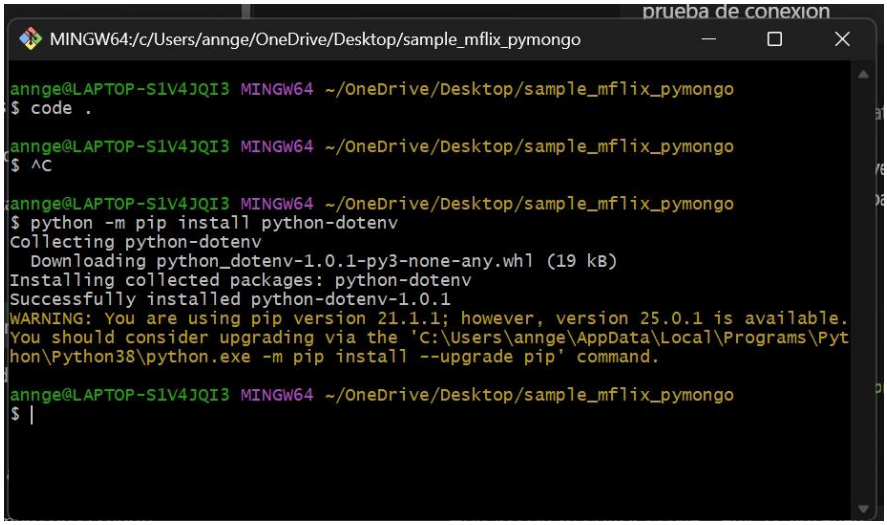
5. Configuración de Python y PyMongo

1. Instalar **Python 3.11 o superior** (en este proyecto se usó 3.14). Es necesario por `asyncio.TaskGroup`.
2. Instalar PyMongo con `python -m pip install pymongo`.



Nota: esta captura es de la primera prueba de instalación, hecha con Python 3.8 y PyMongo 4.10.1. El proyecto necesita Python 3.11 o superior; la versión final usó PyMongo 4.19.

3. Crear la carpeta del proyecto e instalar python-dotenv (lee el archivo `.env` con la conexión). Lo más cómodo es instalar todo junto con `python -m pip install -r requirements.txt`.

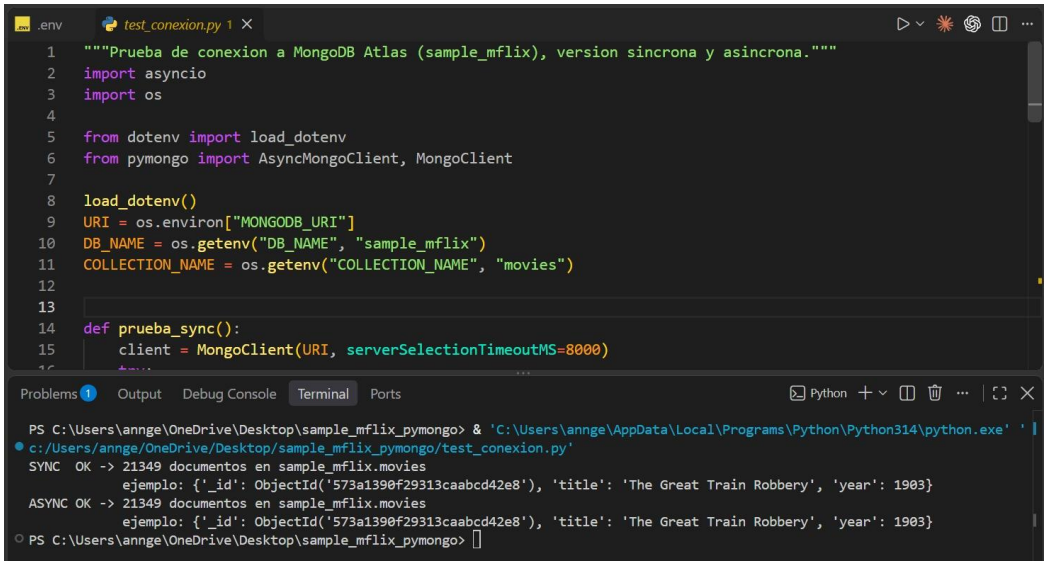


4. Crear el archivo `.env` en la carpeta del proyecto, con la cadena de conexión real (usuario y contraseña, sin comillas):

```
MONGODB_URI=mongodb+srv://USUARIO:PASSWORD@clustersamplemflix.uluz8c0.mongodb.net/
DB_NAME=sample_mflix
COLLECTION_NAME=movies
TEST_COLLECTION=movies_test_TUNOMBRE
```

Cada integrante debe usar un `TEST_COLLECTION` distinto (por ejemplo `movies_test_ana`) para no borrarse los datos entre sí.

5. Ejecutar `python test_conexion.py` antes de empezar. Debe aparecer `SYNC OK` y `ASYNC OK` con 21349 documentos.



6. Si tienes varios Pythons instalados, instala los paquetes con el mismo `python` con el que ejecutas el código (`python -m pip install ...`); si no, aparece `No module named 'dotenv'`.
7. Todo listo para usar el proyecto.

6. Cómo usar las aplicaciones

```
python app_sync.py
python app_async.py
```

Se abre un menú. Se elige un número y la app pregunta los datos que necesite. Las opciones son las mismas en las dos:

Opción	Qué hace
1	Carga masiva: copia películas de `movies` a `movies_test`
2	Crear una película
3	Buscar una película por título (exacto, con mayúsculas y minúsculas)
4	Cambiar el año de una película
5	Eliminar una película
6	CRUD masivo (varias películas)
7	Películas de un año
8	Cuántas películas hay de un género (en inglés: `Comedy`, `Drama`, `Action`...)
9	Las 5 mejor calificadas
10	Los 5 géneros con más películas
11	Las consultas 7, 8, 9 y 10 juntas
12	Prueba de rendimiento (la misma que usa `comparar.py`)
13	Borrar todo lo de `movies_test` (la deja vacía)
0	Salir

Recorrido sugerido para una demostración: opción 13 (dejar `movies_test` vacía), opción 1 (cargar en vivo), opciones 2 a 5 (CRUD de una película), opciones 7 a 10 (consultas) y opción 12 (rendimiento).

7. Comparación de rendimiento

```
python comparar.py
```

Este archivo ejecuta la opción 12 de las dos apps con los mismos datos y parámetros: 2,000 películas en lotes de 200, CRUD masivo de 50 películas (200 operaciones) y 20 consultas (las 4 repetidas 5 veces), con un máximo de 5 operaciones a la vez en la asíncrona. Al final imprime una tabla y **crea el archivo `resultados.md`** en la misma carpeta. Ese archivo no existe hasta que se ejecuta `comparar.py`.

La tabla se copia a la sección 5 de `REPORTE.md`. Conviene ejecutarlo 3 veces y usar el promedio, porque el clúster gratuito y el internet cambian de una corrida a otra.

Los números (2,000, 50, 5) no tienen una razón especial: se pueden cambiar en `config.py`. Solo se buscó tener suficientes operaciones para que se note la diferencia sin llenar el clúster gratuito.

8. Problemas comunes

Error	Causa	Solución
`No module named 'dotenv'`	Paquetes instalados en otro Python	`python -m pip install -r requirements.txt` con el mismo `python` que ejecuta el código
`KeyError: 'MONGODB_URI'`	No encuentra el `.env`	Crearlo en la misma carpeta que los `.py` y revisar que no se llame `.env.txt`
`ServerSelectionTimeoutError` o `SSL handshake failed`	Atlas bloquea la IP	Agregar la IP (o `0.0.0.0/0`) en Network Access, esperar 1 o 2 minutos o probar otra red
`Authentication failed`	Usuario o contraseña incorrectos	Revisar el `.env`; si la contraseña tiene símbolos especiales, codificarlos o usar una más simple
Error con `TaskGroup`	Python menor a 3.11	Instalar Python 3.11 o superior
No existe `AsyncMongoClient`	PyMongo muy viejo	`python -m pip install -U pymongo`
No aparece `resultados.md`	Aún no se ejecutó `comparar.py`	Ejecutar `python comparar.py`

9. Seguridad

- El `.env` tiene contraseñas: **no se sube a GitHub** (ya está en `.gitignore`).
- Usar contraseñas propias para los usuarios de la base de datos, nunca simples como el nombre de usuario.
- `0.0.0.0/0` abre el clúster a cualquier IP: borrarlo en Network Access al terminar el proyecto.
- La colección original `movies` solo se lee; todo lo que se modifica va a `movies_test`.